

Universidade Federal de Viçosa – Campus Florestal
Bacharelado em Ciência da Computação

SISTEMAS DISTRIBUÍDOS

PROFESSOR(A): Thais Regina de M. B. Silva

SISTEMAS DISTRIBUÍDOS ARQUITETURA:

Gabriel Santos Ferreira de Pádua - 4705



Introdução - Arquitetura.....	3
Modelos Físicos.....	4
Tipos de Modelos:.....	4
Tomada de Decisão:.....	4
Justificativa:.....	4
Modelo de Arquitetura.....	5
Paradigma de Comunicação Entre as Entidades Arquitetônicas:.....	5
Modelos de Arquitetura Básicos.....	5
Funções das Entidades:.....	5
Cliente (Papel Ativo):.....	5
Servidor (Papel Passivo e Gerenciador):.....	5
Arquitetura de Camadas Físicas.....	6
Arquitetura de Camadas Lógicas:.....	7
Modelos Fundamentais.....	8
Modelo de Interação:.....	8
Latência:.....	8
Taxa de Transmissão (Largura de Banda):.....	8
Jitter:.....	8
Síncrono ou Assíncrono?:.....	8
Modelo de Falhas:.....	8
Classe Escolhida:.....	8
Descrição da Falha:.....	9
Detecção e Tratamento:.....	9
Modelo de Segurança:.....	9
Proteção aos Processos:.....	9
Proteção ao Canal de Comunicação:.....	9
Conclusão:.....	9

Introdução - Arquitetura

Este documento apresenta a segunda etapa do Trabalho Prático da disciplina de Sistemas Distribuídos, dando continuidade à especificação do Mundo de Construção Livre colaborativo (Horta Minecraft - Garden in Overworld) iniciada na Parte 1.

Enquanto a etapa anterior priorizou o levantamento de requisitos e a definição visual e funcional dos casos de uso, a presente etapa é inteiramente focada na arquitetura do sistema, no pensamento estrutural para sua construção e na sua montagem efetiva. O objetivo central é realizar a modelagem técnica e detalhada do sistema distribuído, estabelecendo as bases de engenharia de software e engenharia de redes necessárias para a futura implementação.

O trabalho descreve as decisões de projeto avaliando o modelo físico mais adequado e definindo a topologia de arquitetura. Através do mapeamento das entidades arquitetônicas, define-se o uso do paradigma e por último consolida o planejamento do sistema ao analisar os modelos fundamentais: interação, falhas e segurança.

Modelos Físicos

Tipos de Modelos:

Abaixo evidencia-se os tipos de modelos físicos que podem ser utilizados na construção de sistemas distribuídos:

<i>Distributed systems:</i>	<i>Early</i>	<i>Internet-scale</i>	<i>Contemporary</i>
<i>Scale</i>	Small	Large	Ultra-large
<i>Heterogeneity</i>	Limited (typically relatively homogenous configurations)	Significant in terms of platforms, languages and middleware	Added dimensions introduced including radically different styles of architecture
<i>Openness</i>	Not a priority	Significant priority with range of standards introduced	Major research challenge with existing standards not yet able to embrace complex systems
<i>Quality of service</i>	In its infancy	Significant priority with range of services introduced	Major research challenge with existing services not yet able to embrace complex systems

Tomada de Decisão:

Para o sistema distribuído em questão, a escolha mais propícia levando em contas condições diversas como ligação dos computadores (internet | Internet), tempo de projeto e infraestrutura disponíveis seria optar pelo mais básico, sendo assim o modelo físico de sistemas distribuídos early, ou “modelo de começo”.

Justificativa:

Como o público-alvo do Sistema Distribuído é bem específico e limitado a sessões de poucos jogadores simultâneos, ele não demanda uma infraestrutura "Internet-scale" ou "Ultra-large". Ele funcionará perfeitamente em uma rede local (LAN) ou em uma rede de pequena escala, onde a heterogeneidade é mais limitada e controlada, com a execução sendo feita por usuários relativamente próximos ou até na mesma inter rede (ex: mesmo wifi).

Modelo de Arquitetura

Paradigma de Comunicação Entre as Entidades Arquitetônicas:

Invocação Remota (troca bilateral): Requisição-resposta (chamada de uma operação).

Tal paradigma faz sentido para o projeto pois o mapa(servidor) deve tanto receber quanto enviar mensagens, assim como o jogador.

Modelos de Arquitetura Básicos

Cliente-servidor (C/S): é uma forma de organizar sistemas em que duas partes têm funções diferentes:

- Cliente: é quem faz a solicitação
- Servidor: é quem recebe a solicitação, processa e envia uma resposta.

Vale salientar que o cliente também pode enviar dados ao servidor mantendo ainda o papel de cliente na comunicação.

Funções das Entidades:

Cliente (Papel Ativo):

Responsável por iniciar as requisições, enviar os comandos e renderizar a interface para o jogador. Executa a invocação dos casos de uso: *UC01* - Identificar-se, *UC02* - Solicitar Visualização do Mapa, *UC03* - Enviar solicitação de Plantio e *UC04* - Enviar solicitação de Colheita. Ele também atua passivamente no *UC05*, apenas recebendo e exibindo a matriz.

Servidor (Papel Passivo e Gerenciador):

Responsável por gerenciar os recursos compartilhados (a matriz da horta). Ele processa o *UC01* gerenciando a lista de sessões (tendo um limite de usuários), valida as regras de negócio de terreno e concorrência nos *UC03* e *UC04*, responde ao *UC02* enviando o estado da matriz e dispara o *UC05* fazendo o broadcast das alterações para todos os clientes conectados.

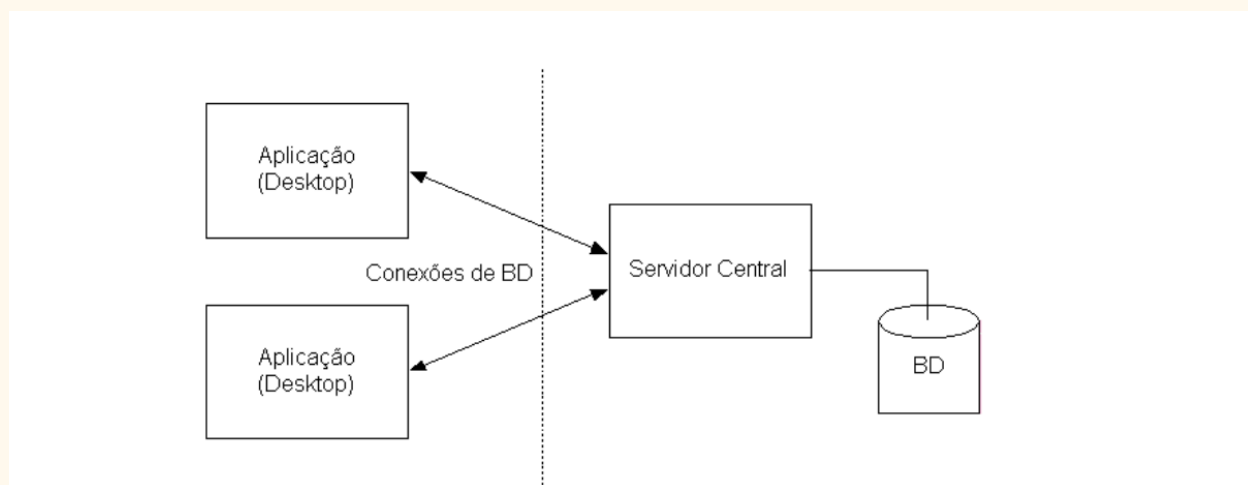
NOTA: há o *UC05* que faz o broadcast das alterações matriciais, e há o *UC02* que pede as alterações matriciais, ambas são necessárias mesmo que pareçam ambíguas, já que a ideia é de tempo em tempo o servidor atualizar os jogadores, mas se algum jogador quiser visualizar antes do intervalo ele pode solicitar

Arquitetura de Camadas Físicas

Como o sistema distribuído possui complexidade baixa, uma configuração de camadas físicas simples seria o ideal.

Mas em um cenário onde esse sistema crescesse, esse pequeno “jogo” teria uma latência muito grande entre os usuários, ou parte deles, e o servidor. Fazendo assim com que a arquitetura tivesse que ser remodelada e ampliada de modo que uma solução seria ter vários servidores interconectados em lugares estratégicos e esses servindo aos clientes (jogadores).

Mas, no contexto deste trabalho prático a seguinte configuração valeria:



- Camada Cliente: Aplicação do usuário rodando no terminal/desktop (interface gráfica/textual).
- Camada Servidor: Servidor central que mantém a lógica do middleware e a estrutura de dados (matriz) em memória.

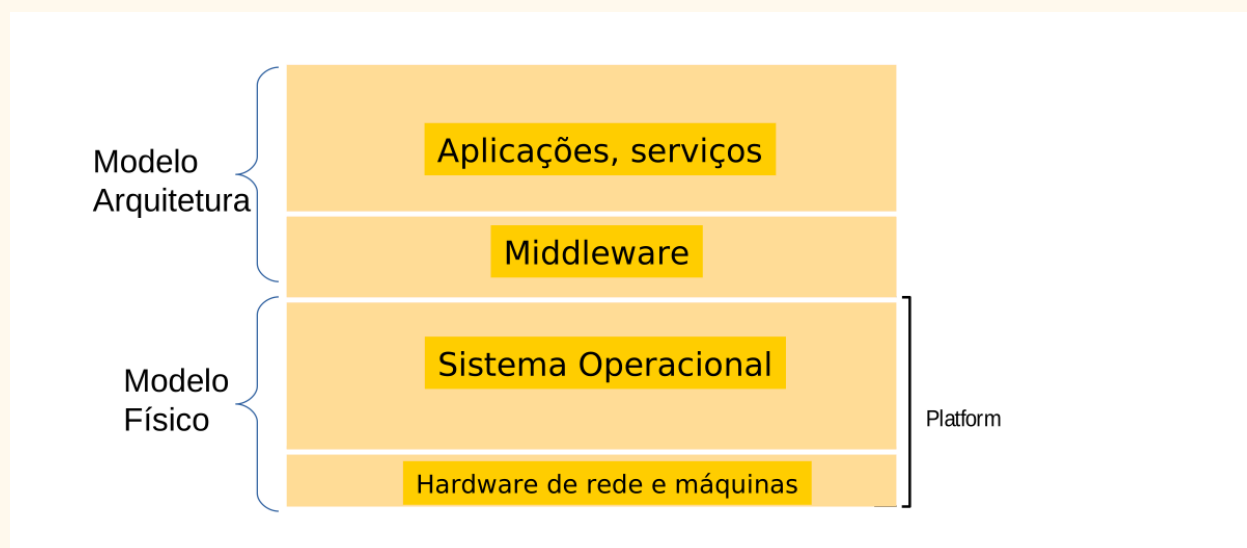
Arquitetura de Camadas Lógicas:

Primeiramente conceituamos o que são as camadas lógicas: Elas são maneiras de separar logicamente o sistema e suas camadas, ou seja, modularizando a parte lógica.

Considerando que haverá posteriormente uma implementação de middleware no trabalho prático, a arquitetura lógica do sistema seguiria o padrão, definindo e separando suas partes de maneira efetiva e sem complexidade desnecessárias.

- Exemplo de complexidade seria adicionar uma camada de criptografia (onde particularmente já ando flertando a algum tempo)

Então, essa seria a arquitetura lógica:



- Aplicações: A lógica visual do jogo (renderização da horta).
- Middleware: O código que gerencia a validação das coordenadas, a sincronização (broadcast) e as conexões simultâneas via threads.
- SO: Ubuntu/Linux gerenciando a rede e as threads.
- Hardware: Placas de rede e processadores executando o jogo.

Modelos Fundamentais

Modelo de Interação:

Latência:

Alta importância. Como é um jogo interativo (mesmo que simples), atrasos na resposta ao tentar plantar algo podem causar a falsa impressão de que a ação falhou. Além de causar um leve incômodo (LAG).

Taxa de Transmissão (Largura de Banda):

Baixa importância. O tráfego consistirá apenas de pequenos pacotes de texto ou *structs* (ex: "Plantar", "X", "Y"), não exigindo conexões de alta capacidade.

Jitter:

Baixa importância. Jitter afeta mais transmissões contínuas (como áudio/vídeo). Para os pacotes discretos de ações na horta, a variação no tempo de entrega não irá quebrar a experiência do usuário.

Síncrono ou Assíncrono?

Assíncrono. No mundo real, os processos clientes terão diferentes velocidades de processamento e atrasos arbitrários de rede podem ocorrer. Não há como dar garantia estrita de limites de tempo de resposta ou de relógios perfeitamente sincronizados entre a máquina de um jogador e o servidor. Só poderia ser Síncrono se fosse possível controlar toda a malha de rede e os processos, assim como seus tempo de máquina.

Modelo de Falhas:

Classe Escolhida:

Falhas por omissão (do processo).

Descrição da Falha:

Um dos jogadores sofre um crash no computador (ex: falta de energia ou fechamento abrupto do terminal), caracterizando uma falha por omissão do processo. O servidor continuaria esperando uma ação desse cliente "fantasma", ocupando indefinidamente 1 das vagas da horta.

Deteção e Tratamento:

A falha pode ser detectada usando timeouts no servidor. Se o servidor não receber um "ping" (sinal de vida) daquele cliente por X segundos, ele trata a falha desconectando o usuário forçosamente e liberando a vaga para um novo jogador.

Modelo de Segurança:

Proteção aos Processos:

Necessária. Um jogador mal-intencionado poderia tentar falsificar sua identidade (spoofing) para se passar por outro jogador e destruir/colher o plantio alheio.

Proteção ao Canal de Comunicação:

Depende do escopo real do jogo.

Tratando-se de um jogo acadêmico/fechado, a criptografia total gera custo computacional desnecessário.

No entanto, sem essa proteção, um usuário na mesma rede local poderia interceptar pacotes e injetar mensagens na rede ("Ameaça à integridade") para realizar ações não autorizadas (trapaças) e até mesmo atrapalhar na comunicação, deixando o jogo impossível de prosseguir.

Conclusão:

Para a especificação de um jogo robusto, implementar mecanismos básicos de proteção de processos como a autenticação no login (protegendo o processo) é vital, mas o canal não necessitaria estritamente de criptografia complexa, a menos que fosse uma aplicação financeira ou comercial.

Porém para ter o mínimo de segurança, sigilo e confiança que ninguém possa atrapalhar nas trocas de mensagem, a segurança do canal de comunicação e a segurança dos processos talvez possa ser uma necessidade de implementação, por mais que não sejam das mais robustas ou rebuscadas